
ALGEBRAIC RETRIEVAL: COMPOSABLE SEARCH FOR AGENTS

A PREPRINT

Damian Delmas

Independent Researcher, Vancouver, BC
damian@getflex.dev

ABSTRACT

Algebraic Retrieval lets AI agents compose search strategies at query time. Relevance criteria, eligibility constraints, and ranking preferences can be expressed together in a mathematical query. The agent can discover available operations, combine them for the question at hand, and revise its strategy after inspecting the results. This gives the agent control over both what it searches for and how it searches. Building on Programmatic Embedding Modulation (PEM), which exposes vector and score arithmetic during retrieval, we demonstrate contrastive scoring, candidate-pool reranking, and weighted ranking as composable queries, alongside executable SQL and PyTerrier counterparts. On the public 11,429-document Vaswani fixture, each program’s implementations select the same document set with score differences below $1e-6$; one tied pair orders differently across scoring paths.

1 Introduction

Retrieval is often presented to an agent as one search call with a query and a few parameters. An agent seeking implementation architecture may also want to suppress marketing material, restrict candidates to engineering discussions, and weight scores by recency. These choices can be written together as one expression.

Algebraic Retrieval lets the agent compose that expression at query time: choose scoring directions, eligible records, and per-record weights. It can revise the program after inspecting results, using named inputs instead of reconstructing their SQL joins or Python setup.

Programmatic Embedding Modulation (PEM) [5] exposes vectors and scores for arithmetic at query time. Algebraic Retrieval gives that surface a mathematical query notation. We compare three programs with SQL and PyTerrier [3], a Python library for composing retrieval pipelines.

2 Preliminaries

An embedding represents a record or query as a vector of numbers in a shared space. Let E hold one record vector per row and q be a query vector. Matrix multiplication $E @ q$ produces one score per record. With unit-length record and query vectors, these scores equal cosine similarity. The scores and their record identities form a scored relation S .

PEM [5] exposes the embedding matrix and score array for arithmetic before selection. A caller can change the query vector before scoring, or add, subtract, and weight scores afterward. SQL filters choose the participating records. Algebraic Retrieval gives these operations a mathematical query notation.

For query vectors in the same embedding space, $+$ and $-$ combine their scores, and scalar $*$ scales a contribution. For example, $(E @ q_1) - 0.5 (E @ q_2)$ favors the first direction while suppressing the second. Selection happens after composition; subtracting two already-truncated top-ten lists would be a different operation. In displayed formulas, q_1 and q_2 denote the source tokens `q1` and `q2`.

A mask m is a set of eligible identities. $m \triangleright S$ removes other records from a scored relation S . A weight relation w supplies a scalar for each eligible identity, and $w \odot S$ multiplies its score. Masks remove records; a zero weight leaves a record present with score zero. Finally, $\tau_{10}(S)$ selects up to ten results, ordered by score descending and identity ascending. The ASCII spelling is `top(10, S)`.

3 Architecture

The runtime uses a materializer: code that executes retrieval operations and stores the results in a temporary SQL table. SQLite resolves the operands, the named inputs to the expression, and NumPy performs vector arithmetic. The parser reads the expression; the planner chooses its execution steps.

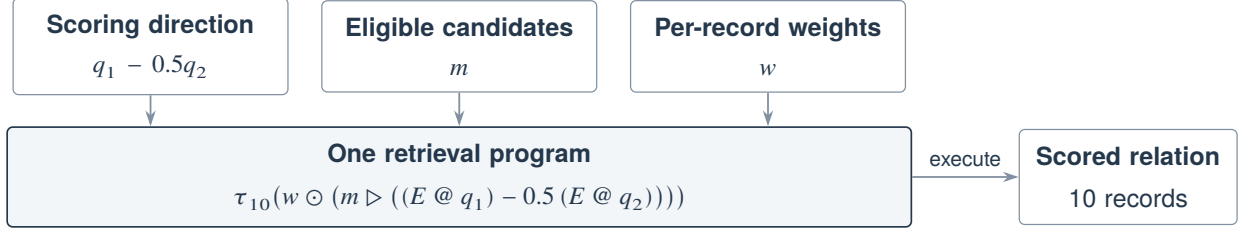


Figure 1: An agent combines scoring directions, a candidate mask, and per-record weights into the query of §5.3. Execution returns a scored relation that can be inspected or composed further.

The agent submits a program through `algebra(' <expression>')`, which can also appear inside a larger SQL query to join results to record text or metadata. Changing a coefficient or adding a mask changes the retrieval strategy through the same call.

Because the query vectors remain explicit in the expression, the planner can combine them before scoring. It rewrites two scoring passes as one using:

$$(E @ q_1) - 0.5 (E @ q_2) = E @ (q_1 - 0.5q_2).$$

The rewrite preserves vector magnitude and uses one matrix multiplication. It is exact in real arithmetic; 32-bit floating-point (`float32`) rounding can cause small differences.

For a fixed E , changing q leaves the scored identities intact. A mask changes which records participate, while per-record weighting generally cannot be absorbed into a change of q .

The SQL listings are executable equivalents, not compiler output. They calculate cosine similarity with the SQLite extension `sqlite-vec` [10], binding `:q1` and `:q2` to the query vectors. They query `embeddings(docno, vector)` and the views `m(id)` and `w(id, weight)` defined in Example bindings.

4 Orient: what the agent sees

Before writing a query, the agent calls:

```
algebra('orient')
```

The response lists available operands, their source relations, and whether they are usable. These six rows come from the Vaswani example:

section	token	domain	status
relation	R	<code>documents(docno, text)</code>	ok
matrix	E	<code>embeddings(docno, vector)</code>	ok
query	q1	<code>query_vectors(qid, vector)</code>	ok
query	q2	<code>embeddings(docno, vector)</code>	ok
mask	m	<code>pyterrier_bm25_results(docno, qid)</code>	ok
weight	w	<code>pyterrier_bm25_results(docno, score, qid)</code>	ok

The response also lists the supported operators. Whoever integrates the corpus declares how each symbol maps to data; the example runner declares the bindings used here. The agent composes expressions over those named inputs.

The examples use the 11,429-document Vaswani collection, with deterministic 128-dimensional vectors. `q1` is the stored query vector for “What are chemical reactions?”; `q2` is a fixed vector from document 2008. BM25 ranks documents by query-term matches; the mask contains the 52 candidates in its recorded PyTerrier result for that query. Each weight is its BM25 score divided by the maximum score in that result. The vectors make the computation reproducible; they are not a learned-embedding retrieval evaluation.

5 Queries the agent can compose

The examples introduce signed scoring and candidate restriction, then combine both with per-record weights. In PyTerrier, `>>` passes results to the next stage and `% 10` keeps the top ten. `D` supplies every document, `M` supplies the recorded BM25 candidates, `A` and `B` score with the two query vectors, and `W` applies the declared weights. `T` orders by score and document identity before cutoff. Their definitions appear at the end of the paper. Reported score differences are relative to Algebra.

5.1 Subtract one similarity search from another

The agent can use a second query or an example document to specify an unwanted direction. Here document 2008 fixes the subtraction operand reproducibly; it is not assigned an irrelevance judgment.

Algebraic expression

$$\tau_{10}((E @ q_1) - 0.5 (E @ q_2))$$

Equivalent SQL

```
SELECT docno AS id,
       (1 - vec_distance_cosine(vector, :q1))
       - 0.5 * (1 - vec_distance_cosine(vector, :q2)) AS score
FROM embeddings
ORDER BY score DESC, id ASC
LIMIT 10;
```

PyTerrier

```
(D >> (A + (-0.5) * B) >> T) % 10
```

PyTerrier adds the first scorer to a negatively weighted second scorer. The SQL alternative calculates two cosine similarities per row; Algebra combines the vectors and scores once. All select the same ten identities, with document 9448 first. PyTerrier returns the same order, with a maximum score difference of $2.24\text{e-}8$. Documents 7429 and 9217 tie exactly in Algebra and PyTerrier, where identity ordering places 7429 first. The SQL path separates them by approximately $5.96\text{e-}8$ and places 9217 first; its maximum returned-score difference is also approximately $5.96\text{e-}8$.

Replacing subtraction with addition blends the two scoring directions.

5.2 Rerank a candidate pool

The agent restricts vector scoring to an existing first-pass result. Here the mask is the 52-document BM25 candidate pool exposed by `orient`.

Algebraic expression

$$\tau_{10}(m \triangleright (E @ q_1))$$

Equivalent SQL

```
SELECT e.docno AS id, 1 - vec_distance_cosine(e.vector, :q1) AS score
FROM embeddings AS e JOIN m ON m.id = e.docno
ORDER BY score DESC, id ASC
LIMIT 10;
```

PyTerrier

```
(M >> A >> T) % 10
```

The candidate pool is scored again and then reduced to ten results. The Algebra planner applies the mask before its vector scoring pass. The SQL join expresses the same allowed identities, and PyTerrier passes those candidates to the scorer. All return the same ordered identities, with document 2417 first. PyTerrier scores match Algebra exactly; the SQL scores have a maximum observed difference of approximately $2.98\text{e-}8$.

5.3 Compose the full retrieval strategy

Within the candidate pool, the agent combines target and suppression scores, applies the declared weights, and selects the top ten. Normalized BM25 weights serve as a mechanical demonstration.

Algebraic expression

$$\tau_{10}(w \odot (m \triangleright ((E @ q_1) - 0.5 (E @ q_2))))$$

Equivalent SQL

```
SELECT e.docno AS id,
       w.weight * ((1 - vec_distance_cosine(e.vector, :q1))
                  - 0.5 * (1 - vec_distance_cosine(e.vector, :q2))) AS score
FROM embeddings AS e
JOIN m ON m.id = e.docno JOIN w ON w.id = e.docno
ORDER BY score DESC, id ASC
LIMIT 10;
```

PyTerrier

```
(M >> (A + (-0.5) * B) >> W >> T) % 10
```

All 52 eligible identities have weights. All three forms return the same ordered identities, with document 10703 first. Maximum score differences relative to Algebra are approximately $2.98\text{e-}8$ for SQL and $1.49\text{e-}8$ for PyTerrier.

The third program composes the choices introduced earlier: combine scoring directions, restrict the candidates, and apply per-record preferences before selection. Its output remains a scored relation that the agent can inspect and reuse.

6 Explanation and evidence

The composition works because intermediate scores remain paired with identities. Arithmetic changes scores, masks determine which records participate, and top-k selects the final result.

The PyTerrier 1.1.2 programs use scores independently computed by FAISS 1.15.0 IndexFlatIP, an exact inner-product scorer. The SQL alternatives use sqlite-vec 0.1.9. All document sets match Algebra and score differences are below $1\text{e-}6$; §5.1 reports the ordering difference. These checks use the recorded upstream BM25 pool and do not compare timing or BM25 implementations.

Score tolerance does not imply rank agreement. Algebra and PyTerrier tie 7429 and 9217 at 0.24868088960647583 , a reference gap of zero; SQL favors 9217 by approximately $5.96\text{e-}8$. Identity ordering resolves only computed ties. If each alternative score differs from its reference by at most ε , a reference gap greater than 2ε preserves their order.

SQL deviations of approximately $2.98\text{e-}8$ also occur in §5.2, which uses a single query vector. A two-pass NumPy calculation of §5.1 preserves the tie, so the ordering difference does not require the rewrite. Stored float32 rows are approximately unit length; sqlite-vec 0.1.9 [10] recomputes vector lengths using float32 accumulation, then rounds cosine distance before SQL subtracts it from one. Reproducing those steps reproduces the pair’s split: a deterministic rounding effect.

The repository’s six-case validation also checks score cutoffs, which retain records that meet a chosen minimum score. Further tests check restriction versus zero-weighting on negative scores and tie ordering, the rewrite in §3, and score cutoffs. A feedback test selects the highest-scoring record, uses its vector as a new query, and scores again. Applying the mask before that selection changes the final ordering, so the planner must preserve where restriction occurs.

PyTerrier composes retrieval stages, called transformers [3], and supports pipeline optimizations. Here the agent writes one expression over discovered operands; the matrix and query vectors remain explicit for rewrites such as §3’s. Earlier work develops vector scoring and query revision from example documents [1,2], and top-k selection over database relations [6,7]. Vespa lets users define document-scoring expressions [8]; LOTUS uses language models to evaluate natural-language conditions on table rows [9]. The experiments do not measure retrieval-quality gains, approximate nearest-neighbor recall, comparative agent query-writing accuracy, or universal compiler correctness.

7 Conclusion

An agent can write retrieval as arithmetic over similarity scores, restrict it with a mask, and change the program at query time. The existing runtime executes these short expressions and returns relations the agent can continue to compose. On the public fixture, three implementations select the same document sets with score differences below $1\text{e-}6$, yet one tied pair orders differently. Numerical agreement must be checked separately from rank agreement.

References

1. G. Salton, A. Wong, and C. S. Yang. “A Vector Space Model for Automatic Indexing.” *Communications of the ACM*, 1975.
2. J. J. Rocchio. “Relevance Feedback in Information Retrieval.” *The SMART Retrieval System*, 1971.
3. Craig Macdonald and Nicola Tonellotto. “Declarative Experimentation in Information Retrieval using PyTerrier.” *ICTIR*, 2020.
4. Sean MacAvaney et al. “Simplified Data Wrangling with `ir_datasets`.” *SIGIR*, 2021.
5. Damian Delmas. “flexvec: SQL Vector Retrieval with Programmatic Embedding Modulation.” arXiv:2603.22587, 2026. DOI: 10.48550/arXiv.2603.22587.
6. Chengkai Li, Kevin C.-C. Chang, Ihab F. Ilyas, and Sumin Song. “RankSQL: Query Algebra and Optimization for Relational Top-k Queries.” *SIGMOD*, 2005. DOI: 10.1145/1066157.1066173.
7. Ronald Fagin, Amnon Lotem, and Moni Naor. “Optimal Aggregation Algorithms for Middleware.” *PODS*, 2001. DOI: 10.1145/375551.375567.
8. Vespa. “YQL Query Language,” “Ranking Expressions,” and “Phased Ranking.” Vespa Documentation, accessed 2026.
9. Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. “Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS.” *PVLDB*, 2025.
10. Alex Garcia. “sqlite-vec: API Reference.” <https://alexgarcia.xyz/sqlite-vec/api-reference.html>. Version 0.1.9 used in the SQL comparisons; cosine implementation.

Example bindings

The bindings use PyTerrier (`pt`), pandas (`pd`), and NumPy (`np`). `ids` lists every document; `score1` and `score2` map identities to their FAISS scores for the two query vectors. `bm25` contains the recorded first-pass result. `topics` contains the single fixture query, passed to each pipeline through `.transform(topics)`.

```
weight = dict(zip(bm25.docno, bm25.score / bm25.score.max()))
D = pt.Transformer.from_df(pd.DataFrame({"qid": "1", "docno": ids}))
M = pt.Transformer.from_df(bm25)
A = pt.apply.doc_score(lambda r: float(score1[r["docno"]]))
B = pt.apply.doc_score(lambda r: float(score2[r["docno"]]))
W = pt.apply.doc_score(lambda r: r["score"] * weight[r["docno"]])

def score_id_order(frame):
    out = frame.sort_values(
        ["score", "docno"], ascending=[False, True], kind="mergesort"
    ).reset_index(drop=True)
    out["rank"] = np.arange(len(out))
    return out

T = pt.apply.generic(score_id_order)
```

The explicit ordering stage supplies the comparison’s tie policy. PyTerrier’s default tied ranks follow input order. Code, fixture instructions [4], and dependencies are available at <https://github.com/algebraicretrieval/algebraicretrieval/tree/main/proofs/pyterrier>. Its `examples.py` runs all three comparisons, the two-pass scoring check, and the scalar rounding reproduction:

```
python proofs/pyterrier/examples.py
```

The native SQL alternatives bind `:q1` to the stored vector for query 1 and `:q2` to document 2008’s embedding, and use these views. SQL’s window maximum and PyTerrier’s `bm25.score.max()` both cover the same 52 rows with `qid='1'`; the maximum is 22.076426496954774, and the normalized weights agree exactly.

```
CREATE TEMP VIEW m AS
SELECT docno AS id FROM pyterrier_bm25_results WHERE qid='1';
CREATE TEMP VIEW w AS
SELECT docno AS id, score / max(score) OVER () AS weight
FROM pyterrier_bm25_results WHERE qid='1';
```